ICT-4202

AI & ML Laboratory Complete Viva + Experiment Guide

All 10 exam sets, 30 practical tasks, essential equations, verified outputs and short answers you can say aloud.

SEARCH + A*	AGENTS + LOGIC	ML + WEKA

What this edition includes

- A fast 180-minute preparation route with experiment-family priorities.
- Plain-English explanations, worked mathematics and interpretable output examples.
- One quick-reference page for each Set 01-10 (A, B, C practical questions).
- Real viva questions reported by classmates, with additional likely follow-ups.
- Offline Python/WEKA checklist and a final one-page formula revision.

Important evidence note

All Python figures and results cited as “from your script” were checked against the supplied code in the uploaded ZIP. The two WEKA tasks require real WEKA GUI runs; no WEKA accuracy figures are invented. Source-based facts and extra conceptual examples are identified separately.

Islamic University, Kushtia | Department of Information and Communication Technology

Prepared for short-notice laboratory and viva revision. Keep your full Python notebooks beside this guide.

Contents and how to use this guide

Use the PDF bookmarks for direct chapter navigation. This compact map helps you choose where to start.

Part	Read it for
Fast preparation	180-minute plan, offline readiness and 10-set mapping.
1. Search	BFS/DFS, DLS/IDDFS, bidirectional, Greedy, UCS, A*, heuristics.
2. Agents + logic	Vacuum, PEAS, chaining, knowledge base and FOL.
3. Classification	k-NN, tree, scaling, information gain and confusion matrix.
4. Regression	Slope, intercept, predictions, worked R^2 and data changes.
5. Clustering	K-Means, centroid, inertia, elbow, silhouette and scaling.
6. WEKA	Actual J48/IBk GUI workflow and 10-fold cross-validation.
Sets 01–10	A/B/C question quick sheets, exact outputs and viva traps.
7. Viva practice	Reported examiner questions and concise model answers.
8–9. Final review	Formula sheet, offline commands and viva checklist.

Jump to your assigned set

Set	Three questions	Set	Three questions
01	Vacuum • Search • k-NN	06	Rules • Search • classifiers
02	Weather • DLS/IDDFS • regression	07	FOL • Greedy • K-Means
03	FOL • A*/UCS • K-Means	08	DLS/IDDFS • WEKA • regression
04	Search • WEKA • regression	09	Search • A* • K-Means
05	Vacuum • Greedy/A* • Decision Tree	10	Rules • Search • classifiers

How to study in the remaining 3 hours

Start with search and regression, because these were probed repeatedly in the reported viva experiences. Next revise K-Means, classification, then agents and logic. Spend the last portion explaining code/output aloud, not copying scripts.

When your assigned set becomes known, jump to its quick sheet first. Then return to the linked common-topic chapter for formulas and detailed explanations.

Fast preparation and exam setup

180-minute practical revision plan

Time	Topic	Specific target
0-15 min	Offline checks	Interpreter, required packages, notebook, plot and WEKA availability.
15-65 min	Search	BFS/DFS, DLS/IDDFS, Greedy, A*, h(n), g(n), path-cost comparisons.
65-100 min	Regression	Slope, intercept, prediction, MSE, R^2 and a changed observation.
100-135 min	ML	k-NN, tree depth, accuracy, K-Means centroid/inertia/silhouette.
135-155 min	Agents and logic	PEAS, NoOp, forward chaining, FOL, implication.
155-170 min	Code and WEKA	Recognize the function that creates each reported result.
170-180 min	Rapid mock viva	Give a 20-second answer to 8-10 questions without reading.

Before the exam: one practical checklist

- Keep **Set_01** through **Set_10** folders together, including the **iris.arff** files in Sets 04 and 08.
- On a connected computer, run `python -m pip install -r requirements.txt`; do not expect downloads to work offline.
- Check imports of numpy, pandas, scipy, matplotlib and sklearn, then run one search script and one ML script.
- Open a Jupyter notebook and confirm its Python kernel can execute cells; confirm plot windows or inline charts are visible.
- For WEKA, check that the Java-based WEKA Explorer actually starts and can load the bundled `iris.arff`.

Expected practical-exam demonstration

For each question, be able to state: (1) task/problem type, (2) input and key parameters, (3) essential algorithm logic, (4) what the output means, and (5) one limitation. The question paper asks for written, executed and demonstrated solutions; permission to use prewritten code should be confirmed.

The 10-set map: your quickest navigation

Set	Q1 / A	Q2 / B	Q3 / C
01	2-room vacuum + PEAS	BFS, DFS, Greedy	k-NN / scaling
02	Weather forward chaining	DLS, IDDFS	Regression / changed score
03	FOL family relations	A* + UCS	K-Means / choose k
04	BFS, DFS, bidirectional	WEKA J48 and IBk	Regression / changed target
05	3-room vacuum	Greedy vs A*	Decision Tree depth
06	Exam-eligibility rules	BFS, DFS, IDDFS	k-NN vs Tree
07	FOL family relations	Greedy / heuristic trap	K-Means / scaling
08	DLS, IDDFS	WEKA + preprocessing	Train/test regression
09	BFS/DFS order	A* / admissibility	K-Means model selection
10	Fan-on rule inference	BFS, Greedy, A*	k-NN vs Tree

1 Search algorithms: concept, trace and comparison

First identify the problem type

A search problem contains an **initial state**, allowed actions or successor states, a **goal test**, and often a **path-cost function**. The graph is the state space. A solution is a sequence of actions from the start to the goal. In these lab tasks, node names are states and directed edges are possible moves.

Expansion means removing a selected node from the frontier and processing or generating its successors. **Frontier/Open** stores candidates waiting to be explored. **Visited/Closed** helps prevent repeated state exploration. “Generated”, “expanded” and “visited” counts depend on the exact implementation; always follow your script’s convention.

Eight algorithms in one table

Method	Select next node by	What to say in viva
BFS	Queue, FIFO	Shallowest in edges; shortest unweighted path, potentially high memory.
DFS	Stack / recursion, LIFO	One branch deeply first; memory-efficient, may miss shorter route.
DLS	DFS with depth ≤ L	Useful bound; can cut off a goal deeper than L.
IDDFS	DLS at L=0,1,2,…	BFS-like shallow completeness; repeats top levels but low memory.
Bidirectional	Forward and backward frontiers	Meet in middle; backward search uses reversed links in directed graphs.
Greedy	Minimum h(n)	Appears near goal; ignores cost already paid; need not be optimal.
UCS	Minimum g(n)	Cheapest accumulated path first; optimal with nonnegative edge costs under standard conditions.
A*	Minimum f(n)=g(n)+h(n)	Balances travel cost and heuristic; optimal under suitable conditions.

BFS versus DFS: the actual Set 09 result

Setting	BFS returned	DFS returned
Original adjacency	A → B → E → G (3 edges)	A → B → D → H → G (4 edges)
Reversed adjacency	A → C → F → G (3 edges)	A → C → F → G (3 edges)

Q Which is better, BFS or DFS?

On the original Set 09 graph, BFS is better **if the target is fewest edges**: three rather than four. DFS may expand fewer nodes on a favorable branch and often uses less memory. If edges have unequal costs, “fewest edges” is not the same as “cheapest path”.

Q Why can DFS change when adjacency order changes?

DFS commits to one branch first, so it is sensitive to the order in which children enter its stack. BFS can also return a different equally short path when neighbors are reordered, while still preserving shortest edge count in an unweighted graph.

Recognize these code operations

Python anchor	What it achieves
`deque / popleft()`	FIFO queue for BFS.
`stack.pop()`	LIFO selection for DFS.
`reversed(neighbors)`	Push order chosen so the first listed DFS child is explored first.
`seen / visited`	Avoid redundant graph-state exploration.

Depth-Limited Search and Iterative Deepening DFS

DLS performs depth-first exploration but refuses to go deeper than the chosen limit. The root has depth 0. **IDDFS** runs DLS at limits 0, 1, 2 and so on until it finds a goal; this is effective when goal depth is not known.

Set 08 experiment	Result from your script
DLS, limit 2	Cutoff. G is beyond the permitted depth.
DLS, limit 3	Success: A → C → F → G.
IDDFS	Success: A → C → F → G.
Number of DLS rounds	Four: limits 0, 1, 2, 3.
Root A visits	4 (one each iteration).

Q Cutoff versus failure?

Cutoff says the depth restriction prevented further exploration; a deeper goal may exist. **Failure** means no solution was found without being blocked by the current limit in the explored finite search space. Set 08 limit 2 produces cutoff, not proof that G is impossible.

Q Why use IDDFS if it repeats nodes?

It does repeated shallow searches, but uses DFS-like memory and can find a shallow solution even when the depth is unknown. For a branching tree, most nodes lie near its deepest level, so the repeated shallow work is often affordable.

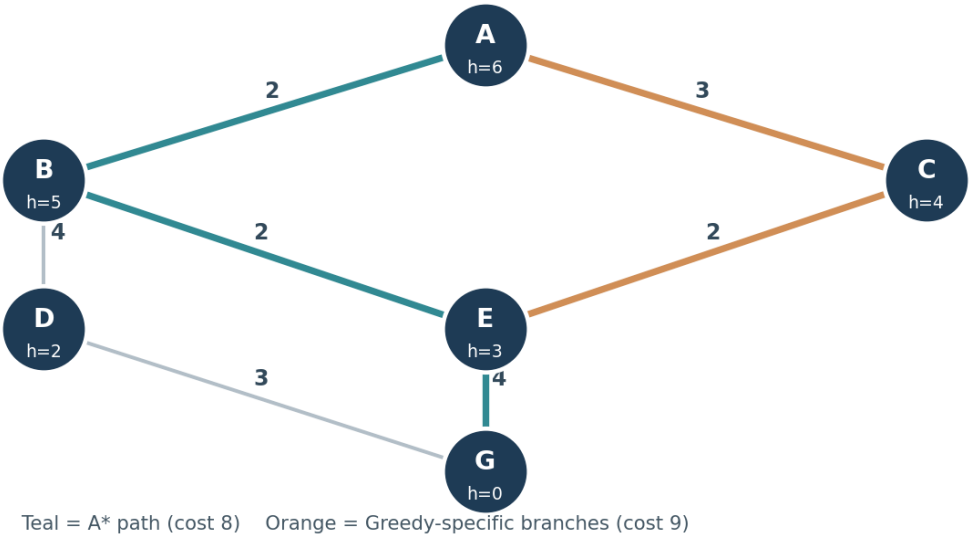
Q What does the depth limit do in code?

The essential condition is `if depth >= limit:` before expanding deeper successors. The IDDFS loop changes `limit` from zero upward and stops when success is returned.

Greedy, UCS and A*: cost and heuristic

Greedy: f(n) = h(n) | UCS: f(n) = g(n) | A*: f(n) = g(n) + h(n)

g(n) = actual cost from the start to n; **h(n)** = estimated least remaining cost from n to a goal; **f(n)** = A* estimate of the total start-to-goal path cost via n. When your question supplies a heuristic table, **use the provided h values**; do not invent another table unless asked.



Set 05: weighted graph and supplied h values. Edge labels are travel costs; colored paths are from the supplied Greedy and A* programs.

Algorithm	Path	Calculation	Total cost
Greedy	A → C → E → G	3 + 2 + 4	9
A*	A → B → E → G	2 + 2 + 4	8

Greedy initially likes C because h(C)=4 is smaller than h(B)=5. A* checks both the cost already paid and the expected remaining cost, so it obtains the better route with cost 8.

How to explain one A* evaluation table

Expanded node	g(n)	h(n)	f(n)=g+h
A	0	6	6
B	2	5	7
C	3	4	7
E	4	3	7
D	6	2	8
G	8	0	8

Q When will Greedy and A* return the same route?

They may agree if the path preferred by the heuristic alone also happens to be a minimum-cost path. Agreement on one graph is possible but not guaranteed. Greedy generally has no optimality guarantee.

Q Where does h(n) come from; how do we calculate it?

A heuristic is constructed from domain information: examples include straight-line distance in route finding and Manhattan distance in grid puzzles. In several laboratory graphs, h(n) is explicitly supplied in the question; in Set 10 you must choose reasonable values yourself.

Admissibility and one misleading heuristic

Admissibility: 0 ≤ h(n) ≤ h*(n), where h*(n) is the true cheapest remaining cost.

In Set 09, the original heuristic has h(A)=5 and true remaining cost h*(A)=6: it does not overestimate. Your script finds the optimal route S → A → D → G with cost **7**.

The script deliberately changes h(A) to **100** (too large). A* then returns S → B → D → G with cost **8**. This example shows how a nonadmissible heuristic may mislead an A* search. The stated guarantee also depends on correct A* implementation and appropriate assumptions for graph search.

Q What if every heuristic is zero?

Then A* priority is f(n)=g(n), which is exactly the selection logic of Uniform-Cost Search. Set 03 demonstrates the same cost 8 route under h=0.

Q What is the cost function in a search problem?

The total path cost is the sum of the costs of its edges. In Set 05: cost(A,B,E,G)=2+2+4=8. A search objective may be fewest edges or minimum weighted cost; these are different objectives.

2 Intelligent agents, knowledge and logic

Intelligent agent, rational agent and PEAS

An **agent** perceives its environment through **sensors** and acts on it through **actuators**. A **rational agent** chooses an action to maximize expected performance given its information; it is not necessarily omniscient. The current percept is the input observed now; the percept sequence is the complete observation history.

PEAS component	Vacuum-world example
P: Performance measure	Rooms cleaned, few unnecessary moves and actions.
E: Environment	Two rooms A/B in Set 01; three rooms A/B/C in Set 05.
A: Actuators	Suck, Left/Right or MoveRight, NoOp.
S: Sensors	Current location and clean/dirty status.

Simple reflex rule: if the current room is dirty, Suck; otherwise move to another room when needed; if every room is clean, return NoOp. A “state” is the current condition of the environment; a “percept” is the sensed information.

Set 01 step	Percept	Action
1	(B, Clean)	Left
2	(A, Dirty)	Suck
3	(A, Clean)	NoOp

Q Why must the vacuum agent eventually choose NoOp?

Once all rooms are clean, further moving or Suck actions bring no cleaning benefit. NoOp is the stop action and prevents needless movement. The supplied Set 01 and Set 05 implementations both terminate after all rooms are clean.

Q What is the difference between simple reflex and knowledge-based agents?

A simple reflex agent relies on condition-action rules and current percepts. A knowledge-based agent stores **facts and rules** and uses an inference mechanism to derive additional facts. It can reason about things that are not directly perceived.

Facts, rules and forward chaining

Forward chaining begins from known facts, checks IF-THEN rules, and adds conclusions whenever every premise of a rule is satisfied. It repeats until no new fact can be added. “Given” facts are asserted initially; “inferred” facts are derived through rules.

IF P ∧ Q THEN R; a rule fires only when BOTH P and Q are known true.

Experiment	Initial facts	Key conclusion
Set 02, weather	Cloudy	Rain → WetGround → Slippery → DriveSlowly.

Experiment	Initial facts	Key conclusion
Set 02, weather + Cold	Cloudy and Cold	WearJacket fires after Rain, because Rain AND Cold hold.
Set 06, exam eligibility	Registered only	Student and HasID, but NOT CanEnterExamHall.
Set 06, with fee	Registered and PaidFee	ExamEligible, then CanEnterExamHall.
Set 10, fan rules	TemperatureHigh only	Hot inferred, but FanOn remains false.
Set 10, fan rules	TemperatureHigh plus Humid OR Rain	Hot and Humid → Uncomfortable → FanOn.

Q Why does WearJacket NOT fire in Set 02 without Cold?

The final rule requires both Rain and Cold. Cloudy can imply Rain, but nothing in the specified rules infers Cold. Inference must not assume a missing premise.

Q What knowledge was given to the agent?

Explain three parts separately: **initial facts** (what the question states as currently true), **rules** (relationships such as Rain → WetGround), and **new facts** inferred by applying the rules. In Set 10, Hot is inferred from TemperatureHigh; it was not an initial fact.

Propositional implication and truth table

P → Q is logically equivalent to ¬P ∨ Q.

P	Q	P → Q
False	False	True
False	True	True
True	False	False
True	True	True

The only time “P implies Q” is false is when P is true but Q is false. This is exactly the truth table your Set 06 script prints. **Forward chaining** uses these implication rules operationally to infer consequences from facts.

First-Order Logic (FOL): family relationships

FOL represents objects, variables, predicates and quantified rules. It expresses relationships more naturally than separate propositional facts for every person. In your Sets 03 and 07, **Parent(x,y)** means x is a parent of y.

Parent(x,y) ∧ Parent(y,z) → Grandparent(x,z)

Parent(p,x) ∧ Parent(p,y) ∧ (x ≠ y) → Sibling(x,y)

Set 03 examples: Parent(Ali,Babu) and Parent(Babu,Karim) infer Grandparent(Ali,Karim). Since Parent(Ali,Babu) and Parent(Ali,Rina), Babu and Rina are siblings. Set 07 similarly infers siblings(Hasan,Nabila) and grandparent(Rahman,Rafi).

FOL symbol	Meaning
∀x	For every x (universal quantifier).
∃x	There exists at least one x (existential quantifier).
∧	Logical AND.
∨	Logical OR.
¬	Logical NOT.
→	Implication: if ... then ...
Predicate	A relation/property such as Parent(Ali,Babu).
Variable / constant	x is a variable; Ali is a named constant.

Q Why should the sibling function exclude self-siblings?

Without x ≠ y a person could be reported as their own sibling. The supplied Python implementation also uses sets and unordered combinations to avoid duplicates such as (Babu,Rina) and (Rina,Babu) being printed twice.

Q Can a Decision Tree do the job of FOL?

Not as a general substitute. FOL uses explicit relational knowledge and deductive rules; a Decision Tree learns predictive feature splits from labeled examples. They can sometimes solve related tasks, but a tree does not automatically provide general quantified relational reasoning.

Tiny code anchors for these experiments

Code expression	Meaning
`if all(status == "Clean" ...)`	NoOp stopping condition for the vacuum world.
`premises.issubset(known)`	A forward-chaining rule is eligible when all premises hold.
`known.add(conclusion)`	Record a newly inferred fact.
`for x,y in parents`	Iterate through Parent(x,y) relationships.
`combinations(children, 2)`	Unique unordered sibling pairs, excluding self-pairs.

3 Machine learning foundations and classification

First say what kind of problem this is

Task	Learning type	Output
Linear regression	Supervised - regression	Continuous number, e.g. estimated score.
k-NN, Decision Tree, J48, IBk	Supervised - classification	Categorical class, e.g. Iris species.
K-Means	Unsupervised - clustering	Cluster number, no training class labels.
BFS/DFS/DLS/A*	State-space search	Path or solution from start to goal.
Knowledge-based agent / FOL	Symbolic reasoning	Inferred facts or logical relationships.

The Iris dataset in your Python code has **150 examples, four numerical features** and three target species (setosa, versicolor, virginica). A stratified 70/30 split in Sets 01/05/06/10 gives 105 training rows and 45 test rows. **Stratified** means the class proportions are maintained in both splits.

k-Nearest Neighbors (k-NN)

k-NN predicts a label for a new observation by identifying its **k closest labeled training examples** according to distance, and usually taking a majority vote. Small k can be noisy, and large k may smooth away useful local distinctions.

$$\textbf{Euclidean distance: } d(x,z) = \sqrt{\left[\ \Sigma\ (x_i - z_i)^2\ \right]}$$

Worked example: the distance from x=(3,2) to centroid or training point z=(2,2) is $\sqrt{[(3-2)^2+(2-2)^2]}=1$. The distance to (6,6) is 5. So the first point is nearer.

k (Set 01)	Test accuracy
1	93.33%
3	95.56%
5	97.78%
7	95.56%

The supplied Set 01 script selects **k=5** based on accuracy on its fixed test split. It achieves 44/45 correct before scaling (97.78%). After standardization, the same k achieves **41/45 (91.11%)** on this particular split.

Q Why does scaling affect k-NN? Why can scaling sometimes lower accuracy?

k-NN depends on distances, so feature units influence which observations are nearest. Standardization changes the geometry; this can improve or worsen a particular held-out result. The Set 01/06 example becomes less accurate after scaling, so do not say scaling “always improves” performance.

Evaluation caution (beyond the basic lab implementation)

Selecting k using the final test set makes its accuracy optimistic for future generalization. A stricter workflow would tune k using cross-validation or a separate validation set, then evaluate once on untouched test data. For the laboratory, still explain the requested experiment as written.

Decision Tree: splitting, entropy and overfitting

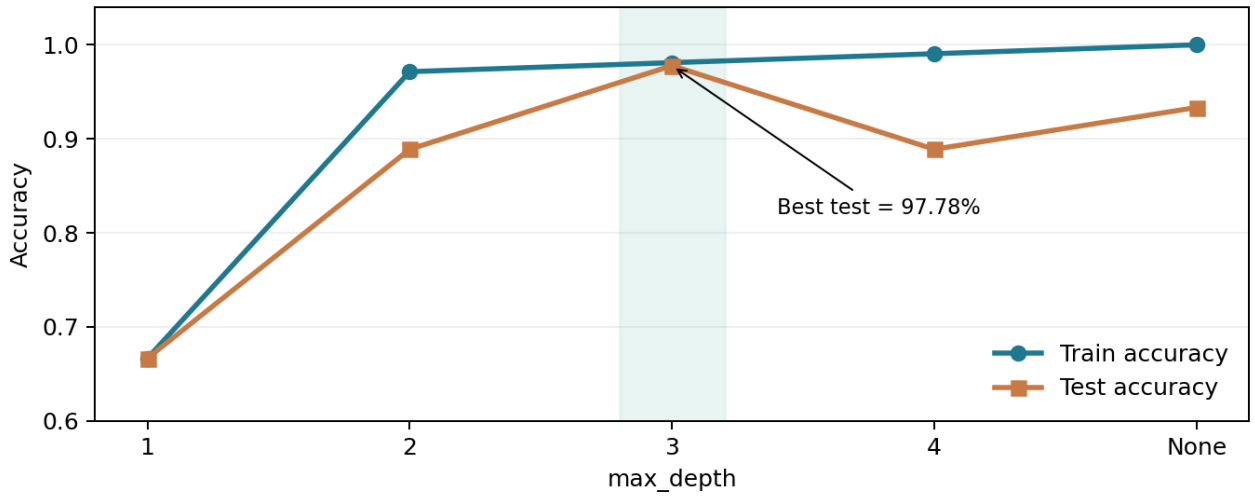
A Decision Tree predicts by making a sequence of feature-threshold decisions. The **root** is the first test, internal nodes are subsequent decisions, and leaves contain predicted classes. Classification trees aim to produce purer groups of class labels after each split.

Entropy $H(S) = -\Sigma\, p_i \log_2(p_i)$ | IG = H(parent) − weighted H(children)

Easy information-gain calculation: in a parent containing 5 positive and 5 negative samples, entropy is 1 bit. Suppose a split creates two groups of five, with class counts (4,1) and (1,4). Each child entropy is approximately 0.7219, so the weighted entropy is 0.7219 and information gain is **1−0.7219 = 0.2781 bit**.

Gini impurity: $Gini(S) = 1 - \Sigma\, p_i^2$

Training a very shallow tree can **underfit** important patterns. Training a very deep tree can **overfit** particular training cases, reducing generalization. The lab changes `max_depth` to show this tradeoff.



Set 05 code output: training accuracy rises with depth, but best held-out accuracy occurs at depth 3 in this specific split.

Tree depth	Train accuracy	Test accuracy
1	66.67%	66.67%
2	97.14%	88.89%
3	98.10%	97.78%
4	99.05%	88.89%
Unlimited	100.00%	93.33%

Q What is the best model in this Decision Tree experiment?

Among the tested max_depth choices on the provided split, depth 3 has the highest test accuracy: 97.78%. Unlimited depth perfectly fits the training data but performs worse on the held-out test set.

Confusion matrix and classifier evaluation

A confusion matrix compares the **actual** and **predicted** labels. In the provided scikit-learn scripts, rows are actual classes and columns are predicted classes. For the best Set 05 tree:

Actual / predicted	Setosa	Versicolor	Virginica
Setosa	15	0	0
Versicolor	0	14	1
Virginica	0	0	15

$$\text{Accuracy} = (TP + TN)/(TP + TN + FP + FN) = 44/45 = 97.78\% \text{ (Set 05)}$$

$$\text{Precision} = TP/(TP + FP);\ \text{Recall} = TP/(TP + FN);\ F1 = 2PR/(P + R)$$

For multiclass Iris classification, TP/FP/FN are defined **one class at a time** (one-vs-rest); macro precision, macro recall and macro F1 average the per-class values. Accuracy only counts overall correct observations. In Set 05 exactly one Versicolor row is predicted as Virginica.

Q How do k-NN and a Decision Tree differ?

k-NN classifies new examples by nearby labeled training data and is distance-sensitive; a Decision Tree learns a sequence of feature thresholds and is usually much less sensitive to uniform feature rescaling. Both are supervised classifiers.

Q What is cross-validation?

In 10-fold cross-validation, data are partitioned into 10 folds. A model trains on nine folds and is evaluated on the remaining fold, repeated so every fold is held out once. The overall score summarizes the held-out predictions.

4 Linear regression: formulas, calculations and results

What regression predicts

Regression is **supervised numerical prediction**. Simple linear regression models the expected mean of a continuous outcome Y from one input X using a straight line. In the study-hours problem, X is the number of study hours and Y is the exam score; the observations in Set 08 are explicitly **synthetic**, not a real student survey.

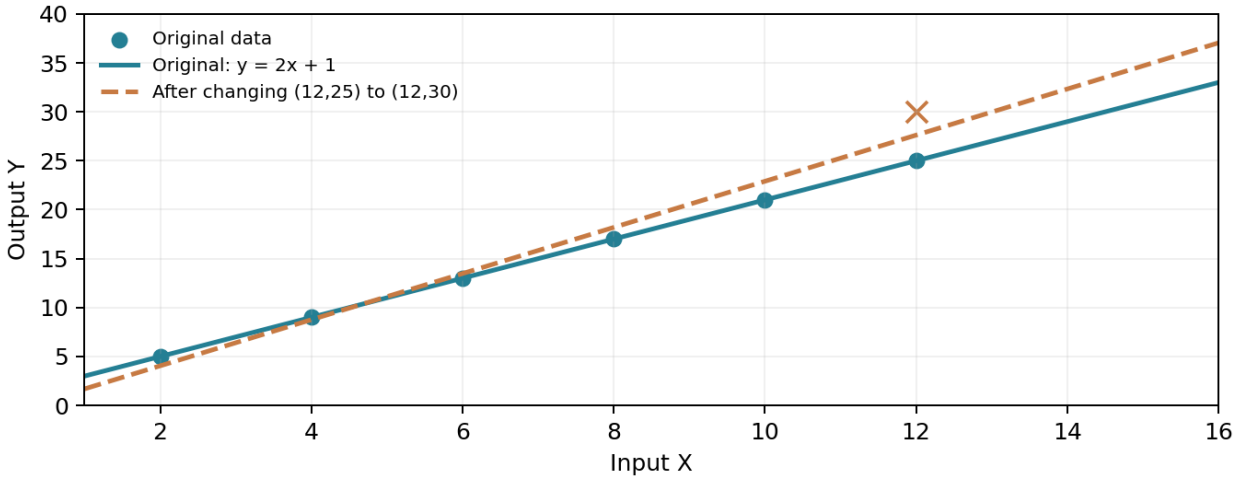
$$\textbf{Fitted line: } \hat{y} = b_0 + b_1 x \textbf{ | residual } e_i = y_i - \hat{y}_i$$

Slope b_1: expected change in predicted Y for one-unit increase in X. **Intercept b_0:** fitted Y when X=0. **Residual:** actual Y minus predicted Y. Ordinary least squares chooses the line minimizing the sum of squared residuals.

$$b_1 = \Sigma (x_i - \bar{x})(y_i - \bar{y}) / \Sigma (x_i - \bar{x})^2 \; ; \; b_0 = \bar{y} - b_1 \bar{x}$$

Worked numerical regression: Set 04

Set 04 supplies X=[2,4,6,8,10,12] and Y=[5,9,13,17,21,25]. Every observation lies exactly on Y=2X+1. Therefore the fitted slope is **2**, intercept is **1**, predicted Y at X=15 is **31**, and R^2 is **1.0000**.



Set 04: changing the final output from 25 to 30 changes the fitted slope/intercept and reduces R^2.

Quantity	Original observations	With (12,25) changed to (12,30)
Slope	2.0000	2.3571
Intercept	1.0000	−0.6667
Training R^2	1.0000	0.9703

Q Why does one modified observation change the regression line?

Least squares fits all observations jointly. A change to one outcome alters the minimization problem, potentially changing the slope, intercept, predictions and R^2. Influential or outlying points can matter substantially.

R^2 and cost function

$$SSE = \Sigma (y_i - \hat{y}_i)^2 \; ; \; SST = \Sigma (y_i - \bar{y})^2 \; ; \; R^2 = 1 - SSE/SST$$

Interpreting R^2: it measures reduction in squared prediction error relative to the constant baseline that predicts the observed mean. In ordinary in-sample regression with an intercept, it is commonly

described as the proportion of observed variation explained by the model. R^2 can be negative on held-out data, and high R^2 does not imply causation.

A small R^2 numerical calculation you can reproduce on paper

Observed y	Prediction ŷ	$(y - \hat{y})^2$	$(y - \bar{y})^2$
2	3	1	4
4	4	0	0
6	5	1	4
Total		SSE=2	SST=8

The mean observation is $\bar{y}=4$. So $R^2=1-2/8=\mathbf{0.75}$. Say: "The model reduces squared error by 75% relative to predicting the mean of the observations."

Regression loss: $MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2$; $RMSE = \sqrt{MSE}$

Q R^2 versus accuracy?

R^2 evaluates numerical regression fit relative to a baseline. Accuracy is the percentage of correct **categorical** predictions in classification. They are not interchangeable metrics.

Q What is an arbitrary equation or expected value?

"Arbitrary equation" is not a distinct named experiment here. If the examiner means a general straight-line relation, explain $y=mx+c$; regression **estimates** its coefficients from data. The expected value of a discrete random variable is $E[Y]=\Sigma\ y \cdot P(Y=y)$. In regression, the fitted line estimates a conditional mean $E[Y|X=x]$, subject to model assumptions.

The three regression scripts: what to memorize

Set	Actual fitted result	Important change or prediction
02	Original slope 1.9714; b_0=1.6000; R^2=0.9786.	Change score at 6 h from 14 to 18 → slope 2.5429; R^2=0.8922.
04	Original slope 2.0000; b_0=1.0000; R^2=1.0000.	Predict at X=15 → 31; modified last target 30 → R^2=0.9703.
08	Slope 4.0211; b_0=11.9432; train R^2=0.9806; test R^2=0.9909.	Predictions: at 7.5 h → 42.101; at 18 h → 84.323.

In Set 08, the synthetic dataset has 16 observations, split into **12 training and 4 test** cases. An 18-hour prediction extends outside the training-time range and is therefore extrapolation, which requires caution.

Code anchors

Code	What to say
`X.reshape(-1, 1)`	Make the one-feature predictor a 2D matrix with one column.
`LinearRegression().fit(X,y)`	Fit slope/intercept using least squares.
`model.coef_[0]`	Estimated slope.
`model.intercept_`	Estimated intercept.
`model.predict([[7]])`	Predicted output at X=7.
`r2_score(y, preds)`	Coefficient of determination.
`train_test_split(...)`	Separate model-fitting observations from evaluation observations.

5 K-Means clustering, inertia and silhouette

How K-Means works: five easy steps

K-Means is an **unsupervised** partitioning algorithm. It clusters points so each observation belongs to the centroid nearest to it, commonly using Euclidean distance. There is no requirement for the known class labels to match the clusters.

- Choose the requested number of clusters k.
- Initialize k centroid positions (the script sets random_state and n_init for reproducibility).
- Assign every point to its nearest centroid.
- Recompute each centroid as the arithmetic mean of its assigned points.
- Repeat assignment and update until changes stop or the iteration limit is reached.

$$\textbf{Centroid: } \mu_j = (1 / |C_j|) \sum (x_i \text{ in cluster } C_j)\ x_i$$

Simple calculation: the points (1,1) and (3,3) have centroid (2,2), because both coordinates are averaged. A point at (3,2) is one unit away from centroid (2,2), but five units from (6,6).

Inertia: the objective minimized by K-Means

$$\textbf{Inertia } J = \sum ||x_i - \mu(\text{cluster}(i))||^2$$

Inertia is the **within-cluster sum of squared distances** to centroids. For the two points (1,1) and (3,3) clustered around (2,2), their squared distances are 2 and 2, so the inertia is **4**. As k increases, inertia normally decreases because the points can be partitioned more finely.

Q Why not choose k only by the lowest inertia?

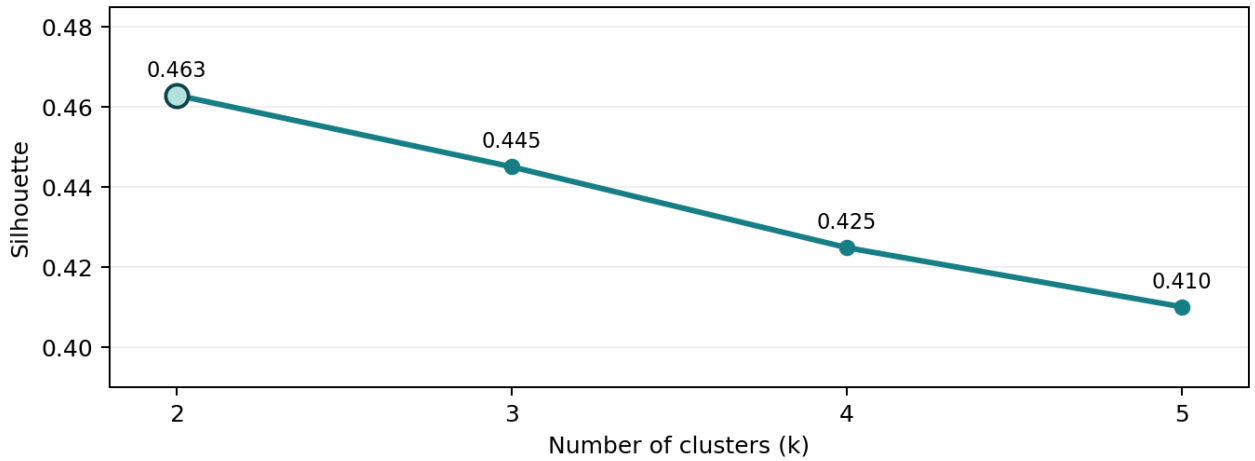
With enough clusters, inertia becomes very small; at k equal to the number of distinct data points it can be zero. A useful k should balance compactness against having too many clusters. Use the elbow plot, silhouette, and application context.

Elbow method versus silhouette score

Elbow method: plot inertia against k and look for the point where the curve begins to flatten; it is a heuristic, not a guaranteed unique mathematical answer. **Silhouette:** compare average within-cluster distance a(i) and nearest-other-cluster distance b(i).

$$\text{Silhouette } s(i) = [b(i) - a(i)] / \max(a(i), b(i))$$

Worked example: if the mean distance to points in the same cluster is a=2 and the mean distance to the nearest other cluster is b=5, then s=(5−2)/5=**0.6**. Positive values near 1 indicate better separation; negative values suggest possible misassignment.



Actual Set 03 / Set 09 results, Iris sepal length and width: highest tested silhouette occurs at k=2.

k	Inertia	Mean silhouette
2	58.2041	0.4630
3	37.0507	0.4451
4	27.9902	0.4249
5	20.9968	0.4101

Q Why does the script choose k=2, even though Iris has three species?

The K-Means experiments use only the first two numerical features and do not use true species labels for fitting. The maximum silhouette among k=2,3,4,5 in those coordinates is at k=2. Clusters need not coincide with known biological classes.

Feature scaling can change cluster geometry

Set 07 fits k=3 using **all four Iris numeric features**, once in original centimeters and once after standardization. It reports the following:

Space	Inertia	Silhouette
Original units (cm)	78.8514	0.5528
Standardized z-scores	139.8205	0.4599

The centroids and assignments can differ after scaling because the distances change. **Do not compare inertia numbers directly across different scales** because distances are expressed in different coordinate systems. Silhouette scores are dimensionless but still describe different feature spaces.

Q What is the difference between k in k-NN and k in K-Means?

In k-NN, k means the number of labeled neighbors that vote on a prediction. In K-Means, k means the number of clusters and centroids to fit. k-NN is supervised; K-Means is unsupervised.

Code anchors

Code	Meaning
`KMeans(n_clusters=k)`	Choose number of clusters.
`model.cluster_centers_`	Learned centroid coordinates.
`model.labels_`	Assigned cluster labels.
`model.inertia_`	Within-cluster squared-distance sum.
`silhouette_score(X, labels)`	Mean silhouette for a fitted partition.
`StandardScaler()`	Feature centering/scaling to z-score coordinates.

6 WEKA practical: what must be done in the GUI

Sets **04-B** and **08-B** require WEKA Explorer. Their supplied Python scripts only check that `iris.arff` contains 150 instances and 5 attributes (4 numeric predictors plus a nominal class). This is **not** a substitute for WEKA classifier results.

Set 04-B: J48 and IBk (k=1,3,5)

- Open **WEKA → Explorer → Preprocess**, load `Set_04/iris.arff`, and select the nominal **class** attribute.
- Under **Classify**, select **trees.J48**, choose **Cross-validation: 10 folds**, and run. Record Correctly Classified Instances (%) and the confusion matrix.
- Use **lazy.IBk** at KNN=1, then 3, then 5, keeping the same 10-fold evaluation settings.

- Compare the best **measured WEKA** IBk configuration against J48. Save the result buffer or screenshots.

Set 08-B: IBk with scaling, then J48

- Load `Set_08/iris.arff`; set class; run **IBk, KNN=5, 10-fold CV**.
- Use **meta.FilteredClassifier** with the **Standardize** (or requested Normalize) filter and inner **IBk, KNN=5**. Run 10-fold CV again.
- Run **J48** under the same evaluation protocol.
- Record actual accuracy/confusion matrix for all runs, and say which **observed** configuration is best.

Why FilteredClassifier is the careful choice

For cross-validation, preprocessing should be learned inside each training fold. Scaling the whole dataset before cross-validation can leak information from held-out folds. Your notebooks recommend WEKA FilteredClassifier for this reason.

Question	Quick answer
What is J48?	A C4.5-style decision tree learner in WEKA.
What is IBk?	A k-nearest-neighbor classifier in WEKA.
What is 10-fold CV?	Ten train/held-out evaluations, each with a different validation fold.
What evidence must I save?	WEKA classifier output/result buffer, accuracy and confusion matrix.
Can I report the Python script’s accuracy as WEKA accuracy?	No. The supplied Python scripts do not run J48/IBk in WEKA.

Set 01 | Simple reflex agent / BFS-DFS-Greedy / k-NN

Scripts: Set_01_A.py, Set_01_B.py and Set_01_C.py. The companion notebook and Set_01_D.py contain supporting explanations/viva practice.

Q1 / A — Two-room vacuum + PEAS

Task: Initial A is Dirty, B is Clean; start at B. Choose movement, Suck and a final NoOp.
Script result: B Clean → Left; A Dirty → Suck; A Clean and all clean → NoOp.
Explain: Use location and dirt as percept; Suck if dirty; otherwise move; stop once both rooms are clean.
Code anchor: `vacuum_policy(); all(...); rooms[location]`

Q2 / B — BFS, DFS and Greedy

Task: Search the supplied directed graph A→G; compare paths and expanded-node counts.
Script result: BFS: A→B→D→G (7 expanded); DFS: same path (4); Greedy: A→C→F→G (4).
Explain: BFS goes breadth-first, DFS follows listed first child, Greedy prioritizes low h(n). This is unweighted, so all displayed paths have three edges.
Code anchor: `deque(); stack.pop(); heapq.heappop()`

Q3 / C — Iris k-NN and scaling

Task: Run k=1,3,5,7 on stratified 70/30 split; compare confusion matrix before/after standardization.
Script result: Accuracy: 93.33%, 95.56%, 97.78%, 95.56%; selected k=5. Scaled k=5 → 91.11%.
Explain: k-NN classifies by nearby labeled observations. Scaling changes distances; in this fixed split it did not improve accuracy.
Code anchor: `KNeighborsClassifier(); StandardScaler()`

Potential viva trap

Do not say Greedy necessarily expands the fewest nodes, or that scaling always improves k-NN. The chosen “best k” here was based on this one test split.

30-second answer to rehearse: “The agent uses location/dirty percepts and stops using NoOp; BFS and DFS explore differently, while Greedy uses h(n). k-NN is supervised, and my best unscaled fixed-split result was k=5 at 97.78%.”

Set 02 | Forward chaining / DLS-IDDFS / regression

Scripts: Set_02_A.py, Set_02_B.py and Set_02_C.py. The companion notebook and Set_02_D.py contain supporting explanations/viva practice.

Q1 / A — Weather knowledge base

Task: Given Cloudy, infer effects. Repeat with Cold explicitly added.
Script result: Without Cold: Rain, WetGround, Slippery, DriveSlowly; no WearJacket. With Cold: WearJacket also inferred.
Explain: WearJacket needs Rain AND Cold. Missing Cold must not be guessed.
Code anchor: `premises.issubset(known); known.add()`

Q2 / B — Depth-limited and iterative deepening

Task: Search S→G at limits 2 and 3, then with IDDFS.
Script result: Limit 2: cutoff. Limit 3 / IDDFS: S→A→D→G. IDDFS visits root S four times.
Explain: Depth of G is three; IDDFS tries limits 0,1,2,3 and repeats shallow work.
Code anchor: `depth_limited_search(); range(max_depth+1)`

Q3 / C — Regression changed last score

Task: Use study hours X=1..6, scores 4,5,8,9,11,14; change the last score to 18.
Script result: Original: slope 1.9714, intercept 1.6000, prediction at 7 = 15.4000, R^2=0.9786. Changed: slope 2.5429, R^2=0.8922.
Explain: The outlying change shifts least-squares estimates and weakens the training fit.
Code anchor: `LinearRegression().fit(); model.score()`

Potential viva trap

A DLS cutoff is not the same as unreachable/failure. Also R^2 is a regression metric, not classification accuracy.

30-second answer to rehearse: “Forward chaining fires only when all antecedents are available. DLS limit 2 is a cutoff and IDDFS finds G at depth 3. Regression predicts numbers; one changed observation alters slope and R^2.”

Set 03 | FOL relations / A*-UCS / K-Means

Scripts: Set_03_A.py, Set_03_B.py and Set_03_C.py. The companion notebook and Set_03_D.py contain supporting explanations/viva practice.

Q1 / A — Family relations in FOL

Task: Implement children, siblings and grandparent relation from Parent pairs.
Script result: Grandparents include (Ali,Karim), (Ali,Sumi), (Babu,Nila); Babu and Rina are siblings.
Explain: Join Parent(x,y) and Parent(y,z) to infer Grandparent(x,z); children sharing a parent are siblings, excluding self-pairs.
Code anchor: `grandparent_relations(); combinations(children,2)`

Q2 / B — A* with zero-heuristic comparison

Task: Search weighted S→G, show node g,h,f and compare with h=0.
Script result: A*: S→B→D→G, cost 8. With h=0, UCS behavior returns same path and cost 8.
Explain: A* minimizes g+h; h=0 reduces it to Uniform-Cost priority g.
Code anchor: `heapq; new_g + h[nxt]`

Q3 / C — K-Means choose k on Iris

Task: Use first two Iris features; k=2,3,4,5; report inertia/silhouette and centroids.
Script result: Silhouette: 0.4630, 0.4451, 0.4249, 0.4101; selected k=2. Centers about (5.224,3.131) and (6.610,2.966).
Explain: Highest tested silhouette favors k=2 for this 2D clustering; it need not match the three known Iris species.
Code anchor: `KMeans(n_clusters=k); silhouette_score()`

Potential viva trap

Do not assume three known species imply k=3 is optimal for unsupervised clustering on just sepal length and width.

30-second answer to rehearse: “I represent relations with predicates, A* evaluates f=g+h, and setting h=0 reproduces UCS. K-Means groups unlabeled points; here k=2 gives the best tested silhouette.”

Set 04 | Bidirectional search / WEKA / regression

Scripts: Set_04_A.py, Set_04_B.py and Set_04_C.py. The companion notebook and Set_04_D.py contain supporting explanations/viva practice.

Q1 / A — BFS, DFS and bidirectional search

Task: On directed P→V, report expansion order, then reverse Q's child order.
Script result: BFS and DFS: P→Q→S→V; bidirectional: same. With Q children reversed, DFS: P→Q→T→V.
Explain: Bidirectional works from start and goal; backward traversal needs reversed directed edges. DFS is sensitive to neighbor order.
Code anchor: `bidirectional_search(); reverse adjacency`

Q2 / B — WEKA J48 versus IBk

Task: Load iris.arff; 10-fold CV; run J48 and IBk k=1,3,5.
Script result: Python dataset check: 150 cases; 5 attributes; 50 in each Iris species. REAL WEKA metrics must be obtained from GUI.
Explain: J48 is a tree; IBk is k-nearest neighbors; save WEKA output buffer and confusion matrices.
Code anchor: `WEKA Explorer / Classify / 10-fold CV`

Q3 / C — Linear regression perturbation

Task: X=2,4,...,12; Y=5,9,...,25; predict X=15; change last target to 30.
Script result: Original: y=2x+1, predict 31, R^2=1. Modified last point: slope 2.3571, intercept −0.6667, R^2=0.9703.
Explain: Changing one target shifts the fitted least-squares line.
Code anchor: `LinearRegression().fit(); predict([[15]])`

Potential viva trap

The Python Set_04_B.py is only an ARFF file check and CANNOT replace WEKA J48 or IBk results.

30-second answer to rehearse: "BFS finds fewest edges, DFS can change with adjacency order, and bidirectional search meets two frontiers. WEKA J48 is a tree and IBk is k-NN; my regression line changes after modifying the last output."

Set 05 | Three-room agent / Greedy-A* / Decision Tree

Scripts: Set_05_A.py, Set_05_B.py and Set_05_C.py. The companion notebook and Set_05_D.py contain supporting explanations/viva practice.

Q1 / A — Three-location cleaning agent

Task: Track A/B/C dirt state, move through the rooms, then NoOp when all clean.
Script result: Illustrative initial: A Dirty, B Clean, C Dirty. Actions Suck → MoveRight → MoveRight → Suck → NoOp.
Explain: At each step the percept is current location + status; NoOp is correct only when every room is clean.
Code anchor: `environment; action = "NoOp"`

Q2 / B — Greedy versus A* weighted search

Task: Use supplied edge costs and h values. Print path and total weighted cost.
Script result: Greedy: A→C→E→G, cost 9. A*: A→B→E→G, cost 8.
Explain: Greedy chooses by h only, A* by g+h; “closer-looking” does not always mean less total cost.
Code anchor: `greedy_weighted(); astar()`

Q3 / C — Decision Tree model depth

Task: Train trees with depth 1,2,3,4,None; report train/test scores and confusion matrix.
Script result: Best test here at depth 3: 44/45 = 97.78%; unlimited depth: 100% train but 93.33% test.
Explain: Depth controls complexity. Deeper trees can overfit the training data.
Code anchor: `DecisionTreeClassifier(max_depth=...)`

Potential viva trap

In an unweighted path, shortest edge count matters; Set 05 edges have weights, so compare total costs. Do not choose the unlimited-depth tree just for 100% training accuracy.

30-second answer to rehearse: “Greedy chose C and cost 9, while A* used g+h and cost 8. For Iris, depth 3 generalized better than an unlimited-depth tree on this fixed split.”

Set 06 | Propositional logic / BFS-DFS-IDDFS / two classifiers

Scripts: Set_06_A.py, Set_06_B.py and Set_06_C.py. The companion notebook and Set_06_D.py contain supporting explanations/viva practice.

Q1 / A — Exam eligibility forward chaining

Task: Infer Student, HasID, ExamEligible and entry permission under two initial-fact scenarios.
Script result: Registered only → Student and HasID, but no exam entry. Registered+PaidFee → ExamEligible → CanEnterExamHall.
Explain: ExamEligible needs BOTH Student and PaidFee. P→Q is false only when P is true and Q is false.
Code anchor: `forward_chain(); implication = (not p) or q`

Q2 / B — BFS, DFS and IDDFS

Task: Search S→G with BFS, DFS and depth iterations.
Script result: All return S→A→C→G. BFS expands 8, DFS expands 4; IDDFS runs limits 0..3 and visits root 4 times.
Explain: BFS, DFS and IDDFS differ in exploration effort, but happen to find the same path in this graph.
Code anchor: `bfs(); dfs(); print_iddfs()`

Q3 / C — k-NN versus Decision Tree

Task: Iris 70/30 split; k-NN k=5, tree depth 3; compare errors and scaled k-NN.
Script result: Unscaled k-NN and Tree both 97.78% test accuracy, with different misclassified rows. Scaled k-NN 91.11%.
Explain: Equal accuracy does not mean identical predictions; scaling changes k-NN distance behavior.
Code anchor: `KNeighborsClassifier(5); DecisionTreeClassifier(3)`

Potential viva trap

Two classifiers can have identical accuracy with different confusion matrices and different mistakes. Always read both.

30-second answer to rehearse: “The fee is a necessary premise for exam eligibility. BFS and DFS differ in expansion strategy. k-NN and the tree both achieved 97.78% here, but made different errors.”

Set 07 | FOL / misleading Greedy / scaled K-Means

Scripts: Set_07_A.py, Set_07_B.py and Set_07_C.py. The companion notebook and Set_07_D.py contain supporting explanations/viva practice.

Q1 / A — FOL and sibling/grandchild functions

Task: Infer children, grandchildren, grandparents and siblings from the Rahman family facts.
Script result: Hasan and Nabila siblings; Rahman grandchildren Rafi and Tania; siblings(Hasan,Hasan)=False.
Explain: Grandparent requires two connected Parent relations; sibling requires shared parent and two distinct people.
Code anchor: `grandchildren_of(); siblings(x,y)`

Q2 / B — Greedy misled by changed h

Task: Run Greedy with original and modified heuristic table.
Script result: Original expanded S,C,F,G (4); modified expanded S,A,C,F,G (5). Returned path is S→C→F→G in BOTH cases.
Explain: Lowering h(A) draws Greedy into a wasted expansion; it still later reaches the same solution.
Code anchor: `greedy_unweighted(); misleading_h`

Q3 / C — K-Means before/after scaling

Task: Fit k=3 using all four Iris features; compare original and standardized spaces.
Script result: Raw: inertia 78.8514, silhouette 0.5528. Standardized: inertia 139.8205, silhouette 0.4599.
Explain: Scaling changes distances and centroids. Raw and scaled inertia cannot be compared directly as equivalent units.
Code anchor: `StandardScaler().fit_transform(); KMeans(3)`

Potential viva trap

Set 07 modified heuristics increase expansions, not path cost/path length in this supplied run; state exactly what the program prints.

30-second answer to rehearse: “Greedy uses only h; a misleading value can waste expansions. K-Means uses Euclidean distances and its inertia changes with feature scaling.”

Set 08 | DLS-IDDFS / WEKA preprocessing / regression

Scripts: Set_08_A.py, Set_08_B.py and Set_08_C.py. The companion notebook and Set_08_D.py contain supporting explanations/viva practice.

Q1 / A — DLS and IDDFS tracing

Task: Run DLS limits 2 and 3 and IDDFS; report cutoff, path and root visits.
Script result: DLS limit 2: cutoff. Limit 3: A→C→F→G. IDDFS finds the same path, root A visited four times.
Explain: Goal is at depth 3. IDDFS repeats limit 0,1,2,3.
Code anchor: `depth_limited_search(); print_iddfs()`

Q2 / B — WEKA IBk scaling vs J48

Task: IBk k=5 with 10-fold CV, repeat with preprocessing, then run J48.
Script result: Python only verifies 150-row iris.arff. Collect all accuracy/confusion matrices from WEKA Explorer itself.
Explain: Use FilteredClassifier for scaling inside CV folds to avoid leakage; J48 is a decision tree.
Code anchor: `meta.FilteredClassifier + lazy.IBk`

Q3 / C — Synthetic train/test regression

Task: Fit study hours to synthetic score on 16 points; train/test split and predict.
Script result: Train 12, test 4; slope 4.0211, intercept 11.9432; train R^2=0.9806, test R^2=0.9909; pred(7.5)=42.101, pred(18)=84.323.
Explain: Slope is predicted score increase per additional hour in this synthetic dataset. Predicting 18 h is extrapolation.
Code anchor: `train_test_split(); LinearRegression(); r2_score()`

Potential viva trap

In Set 08, the regression observations are synthetic and 18 hours is outside the fitted input range. Do not claim the model causally estimates the effect of studying.

30-second answer to rehearse: “DLS has a depth boundary, IDDFS increases it and revisits root four times. Regression is numerical prediction; R^2 measures squared-error improvement versus the mean, not classification accuracy.”

Set 09 | BFS/DFS order / heuristic admissibility / K-Means

Scripts: Set_09_A.py, Set_09_B.py and Set_09_C.py. The companion notebook and Set_09_D.py contain supporting explanations/viva practice.

Q1 / A — BFS and DFS with neighbor reversal

Task: Show A→G path/expansion under original and reversed adjacency.
Script result: Original BFS: A→B→E→G (3 edges); DFS: A→B→D→H→G (4). Reversed: both A→C→F→G (3).
Explain: BFS returns a shallow path; DFS branch choice depends heavily on successor ordering.
Code anchor: `bfs(); dfs(); reversed adjacency`

Q2 / B — A* and admissible h

Task: Compare provided admissible h with deliberately too-large h(A).
Script result: Original h: S→A→D→G, cost 7. Modified h(A)=100: S→B→D→G, cost 8.
Explain: An admissible heuristic does not overestimate true remaining cost. h(A)=100 violates that and can cause a worse result.
Code anchor: `new_g+h[nxt]; best_g`

Q3 / C — K-Means model selection

Task: On first two Iris features, try k=2,3,4,5 and compare inertia/silhouette.
Script result: Silhouette 0.4630, 0.4451, 0.4249, 0.4101; best tested k=2. Inertia falls 58.2041 → 20.9968.
Explain: Inertia decreases as k rises; silhouette reflects cluster separation and compactness.
Code anchor: `inertia_; silhouette_score()`

Potential viva trap

Do not say BFS minimizes weighted cost. Its unweighted guarantee is minimum number of edges; for weighted costs use UCS/A* with suitable assumptions.

30-second answer to rehearse: “BFS is better here for fewest edges, but DFS changes with child order. An admissible h never overestimates true remaining cost; K-Means k can be estimated by elbow/silhouette.”

Set 10 | Integrated KB agent / three searches / classifiers

Scripts: Set_10_A.py, Set_10_B.py and Set_10_C.py. The companion notebook and Set_10_D.py contain supporting explanations/viva practice.

Q1 / A — Fan-on forward chaining

Task: Test TemperatureHigh only, plus Humid, then plus Rain.
Script result: TemperatureHigh alone → Hot only, FanOn false. Adding Humid or Rain eventually infers FanOn.
Explain: TemperatureHigh → Hot; Rain → Humid; Hot AND Humid → Uncomfortable → FanOn.
Code anchor: `forward_chain(initial_facts, rules)`

Q2 / B — BFS, Greedy and A* with chosen weights

Task: Assign positive costs and reasonable h to a directed graph; compare three searches.
Script result: In the script's EXPLICIT illustrative numbers: BFS S→A→C→G cost 17; Greedy S→A→D→G cost 11; A* S→B→D→G cost 5.
Explain: Shortest path in edges need not be cheapest when edge weights differ. Question leaves weights/h to the student.
Code anchor: `bfs(); greedy_weighted(); astar()`

Q3 / C — k-NN versus Decision Tree metrics

Task: Classify Iris with k-NN k=3 and tree max_depth=3; report accuracy and macro metrics.
Script result: k-NN: 95.56% accuracy (2 errors); Tree: 97.78% accuracy (1 error). Tree wins for this fixed run.
Explain: Decision Tree achieves better test accuracy and macro F1 here; not proof it wins on every dataset.
Code anchor: `accuracy_score(); precision_recall_fscore_support()`

Potential viva trap

The numerical weights and heuristic in Set 10-B are NOT specified by the question; your supplied code made one reproducible choice, so another valid choice can produce different results.

30-second answer to rehearse: "The KB agent derives FanOn only after prerequisites become true. BFS counts edges, Greedy uses h, A* uses g+h, so weighted route costs differ. The depth-3 tree wins for this Iris split."

7 Real viva experiences: short answers to practice aloud

These questions follow the three classmates' reported experiences, translated into concise English. Give a definition, one example from your own script, and a reason or limitation. Most answers should take about 15-30 seconds.

Q What kind of problem do DLS and IDDFS solve? Is regression the same?

DLS and IDDFS solve **state-space search** by finding a route from an initial to a goal state. Linear regression is **supervised numerical prediction**; it estimates continuous outcomes rather than finding graph paths.

Q Why would you choose IDDFS over DFS or BFS?

When goal depth is unknown and memory is limited, IDDFS combines repeated depth-limited DFS with the ability to discover a shallow goal. It pays for this with repeated shallow expansions.

Q Does BFS always have a better path than DFS?

BFS guarantees a minimum-edge path for an unweighted graph if implemented properly. DFS may find an equally short path or a longer one, but "better" also depends on memory, expansion count and weighted path cost.

Q What does the heuristic h(n) measure? Is it the true cost?

h(n) is an **estimate** of minimum remaining cost. The true cheapest remaining cost is h*(n). A heuristic is admissible if it never exceeds h*(n). In the exam graph, h values may simply be supplied.

Q What is g(n) and when is f(n) important?

g(n) is the exact cost traveled from the start along the current path; A* ranks candidates by f(n)=g(n)+h(n). Greedy uses only h(n), which can be misleading.

Q Why can Greedy and A* find different routes?

Greedy ignores what it has already spent. A* includes accumulated cost. In Set 05, Greedy costs 9 but A* finds cost 8. They may also agree by coincidence if the heuristic choices lie on an optimal route.

Q What is a cost function? Give three contexts.

A cost function quantifies something to minimize: search path cost is summed edge weights; regression commonly minimizes squared residuals; K-Means minimizes the squared distances to assigned centroids (inertia).

Q What do slope and intercept tell us?

In $\hat{y}=b_0+b_1x$, b_1 is the expected change in predicted outcome when x increases by one; b_0 is the predicted outcome when $x=0$. Neither slope alone nor correlation establishes a causal effect.

Q How do you calculate and interpret R^2?

$R^2=1-SSE/SST$. SSE is squared prediction error and SST is squared deviation from the mean response. $R^2=0.75$ indicates 75% reduction in squared error versus mean prediction in that evaluated sample.

Q Is R^2 allowed to be negative?

Yes. On held-out data, a model can predict worse than the mean-prediction baseline, giving SSE greater than SST and negative R^2.

Q What is an expected value in this context?

In probability, E[Y] is the probability-weighted mean outcome. In regression, the fitted line approximates conditional expected Y at a given X, under model assumptions.

Q How does a Decision Tree choose a split?

A classification tree selects a feature condition that reduces class impurity, such as entropy or Gini. Information gain is parent entropy minus the weighted child entropy.

Q What do we mean by overfitting in the Decision Tree results?

An overfit model fits training cases extremely well but generalizes less well. In Set 05, unlimited depth achieves 100% training accuracy but only 93.33% test accuracy, below depth 3's 97.78%.

Q What is a centroid and a nearest centroid?

A centroid is the arithmetic mean position of points assigned to a cluster. A point is assigned to whichever centroid has the smallest distance according to the model's chosen metric.

Q What does inertia mean in K-Means?

It is the sum of squared distances from each point to its assigned cluster centroid. It decreases or remains unchanged as more clusters are allowed, so raw minimum inertia alone is not a reliable k-choice.

Q How do you decide k in K-Means?

Use the elbow plot of inertia against k, the silhouette score, and application context. In Sets 03 and 09, the tested maximum silhouette is at k=2, even though Iris contains three species.

Q Does scaling always make K-Means or k-NN perform better?

No. Scaling changes distance geometry and can help or hurt the chosen metric on a given dataset. Also, inertia before and after scaling is measured in different coordinate systems and should not be compared directly.

Q What is the difference between a normal agent and a knowledge-based agent?

A basic reflex agent follows percept-based action rules. A KB agent represents facts/rules explicitly and uses an inference mechanism to derive additional facts, such as Rain from Cloudy.

Q What information did the intelligent agent receive in the question?

Distinguish the current percept, the initial state or given facts, the action choices, and any IF-THEN rules. Newly inferred conclusions should be identified separately from facts provided by the question.

Q Why does WearJacket not fire if only Cloudy is known?

Because the rule requires both Rain and Cold. Cloudy can lead to Rain, but nothing supplies Cold unless it is an initial fact. Inferring WearJacket would be unsupported.

Q Can you use a Decision Tree instead of FOL?

A Decision Tree is a data-driven feature-to-label predictor; FOL is a symbolic system for relationships, quantifiers and deductive consequences. A tree can imitate some outputs with data but is not a complete replacement for FOL.

Q What do the precision, recall and F1 values tell us?

Precision is how many predicted positives are correct; recall is how many actual positives are found; F1 combines precision and recall. In multiclass problems these are calculated per class and may be macro-averaged.

Q Why is a confusion matrix helpful if we already know accuracy?

It shows which classes the model confused. In Set 06, k-NN and the tree have the same 97.78% accuracy but make errors on different test rows.

Q What is the purpose of 10-fold cross-validation in WEKA?

It evaluates the model on ten rotated held-out folds to give a more stable comparison than a single split. Keep the class attribute and evaluation protocol the same across J48/IBk runs.

Q Can I say WEKA J48 achieved 97.78% because my Python tree did?

No. That 97.78% is for a specific Python holdout split and decision-tree implementation. The WEKA question explicitly requires running J48 with 10-fold CV and reporting its actual output.

8 Essential equations: one-page quick revision

Concept	Formula / rule	Remember
A* evaluation	$f(n) = g(n) + h(n)$	g = paid cost; h = estimated remaining.
Admissible heuristic	$0 \le h(n) \le h^*(n)$	Never overestimate true cheapest remaining cost.
Path cost	$C(path) = \Sigma$ edge weights	Minimize total weighted cost, not necessarily steps.
Regression	$\hat{y} = b_0 + b_1 x$	Slope = predicted change per x unit.
OLS slope	$b_1 = \Sigma(x-\bar{x})(y-\bar{y}) / \Sigma(x-\bar{x})^2$	Fit slope using data covariance/variation.
OLS intercept	$b_0 = \bar{y} - b_1\bar{x}$	Line passes through $(\bar{x},\bar{y})$.
R^2	$1 - SSE/SST$	Squared-error improvement vs mean baseline.
MSE	$(1/n)\ \Sigma(y-\hat{y})^2$	Regression cost / error.
Euclidean distance	$\sqrt{[\Sigma(x_i-z_i)^2]}$	Used in k-NN and K-Means.
Centroid	$\mu = (1/size)\ \Sigma$ cluster points	Average feature vector.
K-Means inertia	Σ distance^2 to assigned centroid	Smaller as k increases; compare same scaling.
Silhouette	$(b-a) / \max(a,b)$	a=same-cluster, b=nearest-other average distance.
Entropy	$-\Sigma\ p_i \log_2(p_i)$	Zero when class is pure.
Information gain	Parent H − weighted child H	Higher reduction makes a split attractive.
Gini	$1 - \Sigma\ p_i^2$	Another impurity measure.
Accuracy	(TP+TN)/all	Correct predictions / total.
Precision / Recall	TP/(TP+FP) ; TP/(TP+FN)	Predicted positives vs true positives.
F1	2PR/(P+R)	Harmonic mean of precision and recall.
Implication	$P \rightarrow Q \equiv \neg P \vee Q$	False only when P true, Q false.
Grandparent	Parent(x,y) AND Parent(y,z)	Conclude Grandparent(x,z).

The three values to carry into the viva

Set 05: Greedy cost 9, A* cost 8. **Set 08:** DLS limit 2 cutoff; IDDFS root visits 4, regression test R^2 0.9909. **Set 09:** BFS 3 edges versus DFS 4 in original ordering; bad h(A)=100 changes A* cost 7 to 8.

9 Final offline and viva checklist

Commands you can remember (while packages are available)

`python --version` • verify the selected interpreter.
`python -m pip install -r requirements.txt` • install dependencies while online.
`python Set_08/Set_08_A.py` • run a single search program.
`python Set_08/Set_08_C.py` • run regression.
`python Set_08/Set_08_D.py --quiz` • optional self-quiz from the ZIP.

Common exam-day faults and quick actions

Problem	Check first
ModuleNotFoundError	Wrong Python interpreter or a dependency missing in that environment. Test with python -m pip.
Notebook has no output	Choose correct Jupyter kernel, then Run All; check the execution order.
Figure does not display	Check plotting backend or inline mode; plt.show() or savefig() for evidence.
WEKA fails to start	Check WEKA and compatible Java installed before going offline.
WEKA data not loading	Use the actual iris.arff inside Set_04 or Set_08; ensure correct class attribute.
Results differ slightly	Check random_state, n_init, scikit-learn version, neighbor ordering and preprocessing.
DFS path differs	Check successor ordering and the order in which neighbors are pushed to the stack.
R^2 unexpectedly negative	Check whether it is a test score and whether SSE exceeds the evaluation SST.

Last ten minutes: say these aloud

- “Search solves start-to-goal state-space problems; regression predicts continuous outcomes.”
- “BFS uses FIFO, DFS uses LIFO, IDDFS repeats DLS from limit 0.”
- “Greedy uses h(n); A* uses g(n)+h(n); an admissible h does not overestimate.”
- “Linear regression estimates slope/intercept; $R^2=1-SSE/SST$.”
- “K-Means is unsupervised; centroid is a mean; inertia is squared-distance sum; silhouette evaluates clustering.”
- “Decision Tree splits by impurity; very deep trees can overfit.”
- “A knowledge base contains initial facts and rules; forward chaining derives new facts.”
- “Real WEKA evaluation must be performed in WEKA and saved, not inferred from Python.”

Final viva technique

When asked “which method is better?”, always add the criterion: fewest steps, lowest total cost, fewer expanded nodes, less memory, highest held-out accuracy or best silhouette. There is rarely a universal winner without specifying the objective.

Source basis and verification notes

Primary experiment sources: user-supplied *AI_MLQuestionList(3).pdf* (10 examination sets) and *ICT-4202-AI-ML-Laboratory-main(1).zip* (README, thirty A/B/C scripts, ten viva D scripts, ten Jupyter notebooks, iris.arff files). The quick-sheet outputs were reproduced by running A/B/C Python scripts under a headless plotting backend on 12 Oct 2026. Different dependency versions or random tie-breaking can change some results.

Course concept sources: uploaded teaching materials including *ICE-407_Chap3-SEARCH.pdf*, *ICE407_Chap2-Agent.pdf*, *m7-logic.pdf*, *m8-fol.pdf*, *ICT-4201_InfoGain.pdf*, *08ClassBasic.pdf*, *10ClusBasic.pdf*, and *ICT-4201_classifier.pdf*. Additional conceptual mini-examples and interpretation cautions are educational explanation, not exam-paper datasets.

Viva basis: three classmates’ firsthand viva reports supplied in the conversation; their Bangla/Banglish questions were translated and grouped in English. WEKA results were intentionally not fabricated because the ZIP supplies dataset-check scripts, not executed WEKA classifier output.